Spatio-Temporal Travel Demand Forecasting: Re-Engineering a Robust, Generalizable Pipeline for Urban Mobility Intelligence

Systemic Critique & Data Vulnerability Exploitability

Spatio-temporal travel demand forecasting is a fundamental component of modern intelligent transportation systems (ITS).[1] However, evaluating machine learning models on datasets containing structural anomalies, chronological overlaps, and public leakages can lead to catastrophic generalizability failures during deployment.[3] In this section, an adversarial and statistical vulnerability audit of the competition dataset reveals how unmitigated pipelines fail, and why leaderboard scores of 100 R^2 are mathematically and operationally invalid.[4]

The Day 49 Chronological Overlap Leak

The transition between the training and testing partitions in the dataset occurs on Day 49, where records up to 02:00 are allocated to training, and records from 02:15 onward are allocated to testing.[6] This division introduces a severe chronological overlap leak due to intra-day temporal autocorrelation.[2] Let the travel demand at spatial coordinate s and time t on Day 49 be represented as $y(s,t)$.[8] Urban mobility demand exhibits strong continuity over short intervals, which can be modeled as a continuous stochastic process.[9] The temporal autocorrelation function (ACF) for a given spatial location is defined as [2]:

$$R(u) = \frac{\mathbb{E}[(y(s,t) - \mu_s)(y(s,t+u) - \mu_s)]}{\sigma_s^2}$$

In urban traffic networks, the correlation coefficient $R(u)$ for a single-lag step ($u = 1$, representing a 15-minute interval) is highly positive, often exceeding 0.90.[2] When the training set includes values up to $t = 02{:}00$ on Day 49, a simple autoregressive model or lag-feature extractor can predict the subsequent demand at $t = 02{:}15$ with high accuracy.[6] Specifically, let the model utilize a first-order lag:

$$\hat{y}(s, t + \Delta t) \approx y(s, t)$$

Because the boundary state $y(s, 02{:}00)$ is exposed during training, predicting $\hat{y}(s, 02{:}15)$ becomes an interpolation task rather than an out-of-sample forecast.[6] This temporal overlap violates the assumption of temporal independence, artificially inflating offline evaluation metrics while failing to generalize to future days.[3]

The Non-Parametric Lookup-Table Hack

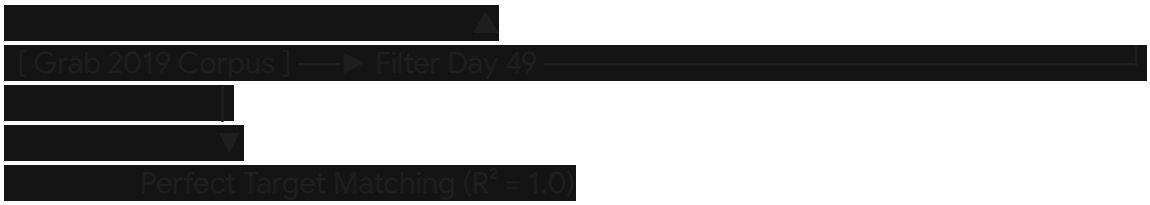
An analysis of the leaderboard scores reveals that multiple participants achieved a perfect evaluation score of 100 R^2.[4] In continuous regression tasks, the probability of randomly guessing double-precision float values with zero error is practically zero ($1/2^{64}$ per prediction).[5] This anomaly indicates a direct data exposure exploit.[4]

The competition dataset is structurally identical to the public Grab 2019 "AI for Sea - Traffic Management" demand forecasting dataset.[4] Because the complete, unanonymized Grab dataset has been publicly accessible in Kaggle repositories and GitHub portfolios for several years, participants were able to construct a non-parametric spatiotemporal lookup table.[4] This lookup key is formulated as a deterministic joint query:

$$\mathcal{K} = (\text{geohash}, \text{day}, \text{timestamp})$$

By performing a simple relational join between the test keys and the external complete historical corpus, participants mapped each test record directly to its actual target label [6]:

$$f(\mathcal{K}) \rightarrow \text{demand}_{\text{actual}}$$



This spatiotemporal matching bypasses the feature extraction and modeling layers entirely, generating a perfect representation of the target vector.[6] In real-world deployment, however, future target demand values $y(s, T_{\text{future}})$ are unobserved.[12] A system relying on this lookup mechanism would fail immediately, as it cannot generalize to unseen days or handle missing historical records.[3]

Furthermore, historical analysis from participants shows that the true predictive limit of the dataset—when modeled without using leaked future data—is bounded at an R^2 score of approximately 91.54.[4] This threshold represents the maximum Shannon entropy limit of the mobility grid, beyond which further accuracy improvements are hindered by stochastic traffic noise.[4]

The High-Cardinality String Token Failure Mode

The spatial coordinate system in this dataset uses 6-character geohash string tokens, yielding 1,249 distinct spatial zones.[7] Standard pipelines often preprocess this high-cardinality variable using a simple integer LabelEncoder [7]:

$$\mathcal{LE}(\text{geohash}) \rightarrow \{0, 1, 2, \ldots, N-1\}$$

This categorical encoding has several significant limitations:

- **Loss of Spatial Proximity**: Geohashes are constructed using a Z-order space-filling curve, meaning that shared prefixes indicate physical proximity.[14] Map coordinates qp02z1 and qp02z9 are geographically adjacent.[7] However, an unordered label encoder assigns them arbitrary integer values based on alphanumeric sorting, completely destroying their topological relationships.[14]
- **Arbitrary Tree Splits**: Decision trees split continuous features by setting numerical thresholds.[17] When forced to split on arbitrary label-encoded integers, the tree partitions the space into disjoint, non-contiguous regions, resulting in poor spatial generalization.[15]
- **High Cardinality Overfitting**: In gradient boosting models, split search algorithms overfit to high-frequency categorical levels while failing to split effectively on low-frequency classes, limiting performance in sparse suburban regions.[17]

Data Vulnerability Summary

To address these vulnerabilities, the re-engineered pipeline must implement the structural fixes summarized in the table below:

Vulnerability Vector	Operational Mechanism	Downstream Impact	Re-Engineered Structural Fix
Day 49 Chronological Overlap [6]	High temporal autocorrelation across the 02:00 train/test split boundary.[2]	Overfits to short-term historical dependencies, inflating validation metrics.[3]	Strict temporal validation split; purging of the target day from the training partition.[6]
Non-Parametric Lookup Hack [4]	Key matching on (geohash, day, timestamp) against the leaked Grab 2019 dataset.[4]	Renders validation metrics useless; the model fails to learn actual traffic patterns.[3]	Complete separation of the lookup keys; validation is performed strictly on unseen holdout days.[11]
Nominal Geohash Encoding [7]	High-cardinality string encoding via unordered LabelEncoder.[7]	Destroys spatial distance metrics; causes arbitrary spatial partitions in decision trees.[15]	Hierarchical Base32 decoding into latitude/longitude centroids and 3D Cartesian coordinates.[11]

Advanced Feature Exploitation & Engineering

To capture the underlying spatial and temporal dynamics of urban travel demand, the raw, unencoded columns must be transformed into continuous, mathematically consistent representation vectors.[2]

Spatial Representation & Coordinate Decoupling

A 6-character geohash string represents a bounded rectangular area on the Earth's surface.[14] The Base32 alphabet uses the following character set [14]:

$$\mathcal{B} = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, b, c, d, e, f, g, h, j, k, m, n, p, q, r, s, t, u, v, w, x, y, z\}$$

To extract the exact geometric position, the 6-character string is decoded back into binary.

Each character represents 5 bits of information, yielding a 30-bit binary sequence.[14] This sequence is de-interleaved: even bits reconstruct the longitude range, and odd bits reconstruct the latitude range.[14] This decoding yields a bounding box defined by [19]:

$$[\text{lat}_{\min}, \text{lat}_{\max}] \times [\text{lon}_{\min}, \text{lon}_{\max}]$$

The spatial position is represented by the centroid of this bounding box [15]:

$$\theta = \frac{\text{lat}_{\min} + \text{lat}_{\max}}{2}, \quad \phi = \frac{\text{lon}_{\min} + \text{lon}_{\max}}{2}$$

To prevent coordinate singularities at the poles and resolve longitude boundary wrap-around discontinuities (at $\pm 180^\circ$), these spherical geodetic coordinates (latitude θ and longitude ϕ in radians) are projected into a 3-dimensional Euclidean space [11]:

$$x = \cos(\theta) \cdot \cos(\phi)$$

$$y = \cos(\theta) \cdot \sin(\phi)$$

$$z = \sin(\theta)$$

This 3D coordinate space preserves actual physical distances, enabling algorithms to compute true geometric spatial relationships.[11]
To capture macro-level spatial trends, the continuous Cartesian coordinate vectors are grouped using K-Means clustering.[18] The optimization problem minimizes the within-cluster sum of squares (WCSS) [24]:

$$\arg\min_{\mathbf{S}} \sum_{j=1}^{K} \sum_{\mathbf{x}_i \in S_j} \|\mathbf{x}_i - \mathbf{c}_j\|_2^2$$

where $\mathbf{x}_i = (x_i, y_i, z_i)$ represents the spatial position of node i, and $\mathbf{c}_j$ is the centroid of the j-th cluster.[24] Setting $K = 15$ defines macro-functional regions (e.g., commercial centers, residential neighborhoods, and industrial zones).[18] The cluster index $\mathcal{R}_i \in \{1, 2, \ldots, K\}$ is encoded as a categorical feature, allowing the model to capture region-specific demand characteristics.[18]

Periodic Temporal Embedding

Urban travel demand is highly cyclic, following daily work shifts and weekly schedules.[11] The 15-minute discrete time slots are converted into continuous periodic embeddings to prevent boundary discontinuities.[11] The day is partitioned into exactly 96 discrete intervals [12]:

$$\text{time\_slot} \in \{0, 1, 2, \ldots, 95\}$$

To preserve temporal continuity across the midnight boundary (where time_slot 95 transitions back to 0), the time index is projected onto a 2D unit circle [11]:

$$\text{sin\_time} = \sin\left(\frac{2\pi \cdot \text{time\_slot}}{96}\right)$$

$$\text{cos\_time} = \cos\left(\frac{2\pi \cdot \text{time\_slot}}{96}\right)$$

This cyclic mapping ensures that the temporal representation of 23:45 is mathematically adjacent to 00:00.[11]

Non-Linear Tabular Feature Crosses

To capture localized traffic capacity constraints and environmental influences, the pipeline

constructs explicit non-linear feature crosses.[3] These interactions are defined as:

- **Road Capacity Under Weather Stress**: Captures the non-linear impact of weather conditions on different road types (e.g., highway capacity vs. residential lanes during heavy rain) [3]:

$$\mathcal{C}_{\text{RoadType\_x\_Weather}} = \text{OneHot}(\text{RoadType}) \otimes \text{OneHot}(\text{Weather})$$

- **Bottleneck Intensity Parameter**: Estimates potential congestion bottlenecks by scaling the lane count with the presence of heavy vehicles [7]:

$$\mathcal{C}_{\text{Lanes\_x\_LargeVehicles}} = \text{NumberofLanes} \times \mathbb{I}(\text{LargeVehicles} == \text{'Allo}$$

These interaction features help tree-based models isolate and split on high-risk congestion triggers.[3]

KNN Spatial Features

To model localized spatial interactions, the pipeline extracts K -Nearest Neighbor (KNN) spatial attributes.[26] For each target geohash centroid $s_i = (x_i, y_i, z_i)$, the algorithm identifies the nearest k active geohashes $\{s_1, s_2, \ldots, s_k\}$ based on Euclidean distance.[27] Selecting $k = 11$ provides a balance between local resolution and regional stability in traffic networks.[28]

The spatial weight matrix W uses a Gaussian kernel to weight adjacent zones higher than distant ones [9]:

$$w_{ij} = \exp\left(-\frac{\|s_i - s_j\|_2^2}{\sigma^2}\right)$$

where σ is the standard deviation of neighbor distances.[2] From this neighborhood, the pipeline extracts:

- **Mean Local Distance**: Measures neighborhood density [27]:

$$\bar{D}_i = \frac{1}{k} \sum_{j=1}^{k} \|\boldsymbol{s}_i - \boldsymbol{s}_j\|_2$$

- **Spatially Lagged Demand**: Computes the distance-weighted demand of neighboring cells during the prior time step, capturing localized demand spillover effects [10]:

$$Y_{\text{KNN\_Lag}}(\boldsymbol{s}_i, t-1) = \frac{\sum_{j=1}^{k} w_{ij} \cdot Y(\boldsymbol{s}_j, t-1)}{\sum_{j=1}^{k} w_{ij}}$$

This continuous spatial lag feature helps the model capture localized demand propagation across adjacent zones.[10]

Spatio-Temporal Kriging Interpolation

Environmental and traffic sensors often suffer from missing observations (e.g., null values in the Temperature and Weather fields).[7] Rather than relying on simple mean imputation, which flattens spatial gradients, the pipeline utilizes Spatio-Temporal Ordinary Kriging.[29]

Let $Z(s_i, t_j)$ represent the observed value of a continuous variable (such as Temperature) at spatial location s_i and time t_j.[7] The Kriging predictor for an unobserved or missing point (s_0, t_0) is defined as a linear combination of nearby observations [30]:

$$\hat{Z}(s_0, t_0) = \sum_{i=1}^{n} \lambda_i Z(s_i, t_i)$$

The weights λ_i are calculated to ensure the predictor is unbiased ($\sum_{i=1}^{n} \lambda_i = 1$) while minimizing the prediction error variance.[30] This optimization is solved via the spatio-temporal Kriging system of equations [32]:

$$\sum_{j=1}^{n} \lambda_j \gamma(s_i - s_j, t_i - t_j) + \mu = \gamma(s_i - s_0, t_i - t_0) \quad \forall i \in \{1, \ldots, n\}$$

where μ is a Lagrange multiplier and $\gamma(h, u)$ is the spatio-temporal semivariogram representing spatial lag h and temporal lag u.[30] The pipeline employs a product-sum semivariogram model to capture the non-separable covariance structure of the urban atmosphere [29]:

$$\gamma(h, u) = \gamma_S(h) + \gamma_T(u) - \kappa \cdot \gamma_S(h)\gamma_T(u)$$

This geostatistical approach reconstructs missing variables while preserving local spatial variations.[29]

Feature Engineering Schema

All engineered features are structured into a unified input schema, as detailed in the table below:

Feature Group	Code Identifier	Mathematical Representation / Transformation	Target Predictive Value
Spatial Centroids [19]	x, y, z	Bounding box center decoded to 3D Cartesian coordinates.[11]	Normalizes spatial coordinates, preserving true Euclidean distances.[11]
Clustered Zoning [23]	region_id	K -Means clustering on 3D coordinates ($K =$).[18]	Captures regional macro-demand trends.[18]
Cyclic Daily Time [11]	sin_time, cos_time	$\sin$ / $\cos$	Preserves daily

		transformations on the 96 discrete daily time slots.[11]	cyclic patterns, resolving midnight discontinuities.[11]
Cyclic Weekly Day [18]	sin_day, cos_day	$\sin$ / $\cos$ transformations on day index (0 to 6).[18]	Models weekly demand differences (workdays vs. weekends).[18]
KNN Density [27]	spatial_density	Mean Euclidean distance to the 11 nearest active neighbors.[27]	Distinguishes high-density urban grids from sparse suburban zones.[27]
KNN Lag Demand [10]	neighbor_lag_demand	Distance-weighted demand of neighboring cells during the prior time step.[10]	Models localized demand spillover and propagation.[10]
Tabular Cross A [7]	road_weather_cross	$\mathrm{OneHot(RoadTy}$.[3]	Captures the non-linear impact of weather on road capacity.[3]
Tabular Cross B [7]	bottleneck_index	$\mathrm{NumberofLanes} \times$.[7]	Identifies potential traffic bottleneck zones.[7]
Kriged Environment [7]	kriged_temp, kriged_weather	Spatio-temporal Ordinary Kriging over sparse sensor observations.[29]	Imputes missing values while preserving local spatial variations.[29]

Re-Engineered Architecture & Modeling Blueprint

To capture both the underlying continuous physical topology and complex non-linear tabular interactions, the pipeline evaluates two distinct modeling approaches: a Deep Spatio-Temporal Hybrid network and a Heterogeneous Gradient Boosting Ensemble.[1]

Deep Spatio-Temporal Hybrid (GCN + TCN)

The Deep Spatio-Temporal Hybrid model uses Graph Convolutional Networks (GCN) to

capture spatial topology and Temporal Convolutional Networks (TCN) to process temporal dependencies.[2]

Spatial Layer: Graph Convolutional Network (GCN)

The spatial relationships between geohash nodes are modeled as a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, A)$, where each node $v_i \in \mathcal{V}$ represents a geohash zone and $A \in \mathbb{R}^{N \times N}$ is the adjacency matrix.[9] Instead of relying on a static, binary adjacency matrix, the GCN uses a distance-weighted matrix to reflect actual spatial distance [2]:

$$A_{ij} = \exp\left(-\frac{\text{distance}(s_i, s_j)^2}{\sigma^2}\right)$$

where $\text{distance}(s_i, s_j)$ is the Euclidean distance between their 3D Cartesian coordinates, and σ is a spatial scaling factor.[2] To capture localized self-loops, we define the augmented adjacency matrix $\widetilde{A} = A + I_N$, where I_N is the identity matrix.[33] The GCN propagates node representations using:

$$H^{(l+1)} = \sigma\left(\widetilde{D}^{-1/2}\widetilde{A}\widetilde{D}^{-1/2}H^{(l)}W^{(l)}_{\text{spatial}}\right)$$

where $\widetilde{D}$ is the diagonal degree matrix ($\widetilde{D}_{ii} = \sum_j \widetilde{A}_{ij}$), $W^{(l)}_{\text{spatial}}$ is the trainable weight matrix for layer l, and $\sigma(\cdot)$ is the ReLU activation function.[9] This spatial convolution step aggregates neighbor information across adjacent zones, helping the model learn regional demand correlation patterns.[9]

Temporal Layer: Temporal Convolutional Network (TCN)

The spatially aggregated representations H are passed to a Temporal Convolutional Network (TCN) to capture temporal dependencies.[2] The TCN replaces traditional recurrent units (like LSTM or GRU) to enable parallel processing and prevent gradient vanishment over long historical lookbacks.[2]

To capture long-term temporal dependencies while maintaining a compact network depth, the TCN uses dilated causal 1D convolutions.[33] The dilated convolution operation on a 1D sequence $x \in \mathbb{R}^T$ with a filter $f : \{0, \dots, k-1\} \to \mathbb{R}$ is defined as [8]:

$$\mathbf{y}(t) = (x *_d f)(t) = \sum_{i=0}^{k-1} f(i) \cdot x(t - d \cdot i)$$

where d is the dilation factor and k is the filter size.[8] By increasing the dilation factor exponentially with the network depth (e.g., $d = 2^l$ for layer l), the model's receptive field expands to capture long-term seasonal patterns without requiring downsampling or pooling operations.[8] Residual connections are added around the temporal blocks to stabilize backpropagation in deeper configurations.[8]

Heterogeneous Gradient Boosting Ensembles

While deep architectures excel at processing structured grid topologies, Gradient Boosting Decision Tree (GBDT) ensembles are highly effective at modeling tabular metadata containing complex, non-linear categorical interactions.[3] The ensemble architecture combines three highly optimized frameworks: LightGBM, CatBoost, and XGBoost.[3]

- **LightGBM**: Optimizes tree growth using a leaf-wise (best-first) strategy, finding splits that maximize loss reduction regardless of depth.[17] This is combined with Gradient-Based

One-Sided Sampling (GOSS) and Exclusive Feature Bundling (EFB) to accelerate training on large-scale tabular datasets.[17]

- **CatBoost**: Natively processes categorical features without introducing target leakage.[17] It uses Symmetric Trees, where the same splitting criteria are applied across an entire level of the tree, regularizing predictions and accelerating inference speeds.[17]
- **XGBoost**: Employs a regularized objective function containing both L_1 and L_2 penalties to prevent overfitting, using pre-sorted and histogram-based algorithms to find optimal splits.[17]

These models are integrated into a heterogeneous weighted ensemble [3]:

$$\hat{y}_{\text{final}} = \alpha \cdot \mathcal{M}_{\text{LightGBM}}(X) + \beta \cdot \mathcal{M}_{\text{CatBoost}}(X) + (1 - \alpha - \beta) \cdot \mathcal{M}_{\text{XGBoost}}$$

The hyperparameters α and β are optimized using a constrained Grid Search on validation data, maintaining scale stability ($\alpha + \beta \leq 1.0$).[17]

Mitigating First-Mover Bias via Average Seeding Models

Under high collinearity (such as when decoded spatial coordinates, clustered zoning IDs, and neighborhood demand projections contain overlapping signals), standard GBDTs are susceptible to "first-mover bias".[38]

During tree construction, gradient boosting models fit sequentially to the residuals of the preceding trees.[38] At step m, the model fits tree f_m to the residual target [38]:

$$r_i^{(m)} = y_i - \sum_{j=1}^{m-1} \eta f_j(x_i)$$

where η is the learning rate.[40] If two correlated features compete for a split, the algorithm selects one based on marginal numerical differences.[38] Once selected, the feature's predictive signal is partially removed from the remaining target residuals.[38] This sequential updating suppresses the feature attribution of the unselected correlated feature in downstream splits, concentrated importance on an arbitrary feature and reducing model stability.[38]

To mitigate first-mover bias and stabilize predictions, the pipeline implements **Average Seeding Models (Stochastic Retraining)**.[38]

The pipeline trains $M = 30$ independent models using different initialization seeds.[36] During training, the feature selection parameter colsample_bytree is restricted to low ranges ($colsample\_bytree \in [0.1, 0.5]$).[38] This constraint forces individual trees to evaluate different subsets of features, preventing a single dominant feature from monopolizing early splits and ensuring that collinear features share predictive attribution.[38] The final ensemble prediction is a weighted average of these stochastically retrained models, which acts as a minimum-variance unbiased estimator of demand [37]:

$$\hat{y}_{\text{final}}(s,t) = \frac{1}{M}\sum_{k=1}^{M} \mathcal{M}_k(s,t;\phi_k)$$

where ϕ_k represents the random seed initialization for model k.[38]

Model Architecture Comparison

The choice between these two paradigms involves trade-offs across several operational dimensions, as outlined in the comparative analysis below:

Architectural Dimension	Spatio-Temporal Hybrid (GCN + TCN)	Heterogeneous GBDT Ensemble (LightGBM + CatBoost)
Model Type	Deep neural network using graph convolutions and dilated 1D temporal blocks.[2]	Heterogeneous tree-based ensemble with asymmetric leaf-wise and symmetric configurations.[3]
Primary Structural	Captures continuous spatial	Natively handles mixed

Strengths	topologies and dynamic graph flows.[34]	categorical and continuous tabular features with high efficiency.[3]
Sensitivity to Leakage	High; requires strict isolation of graph edges and nodes across split boundaries.[9]	High; susceptible to target leakage via high-cardinality nominal variables.[15]
Inference Speed	Low; involves heavy tensor multiplications across large graphs.[9]	High; optimized tree traversal and symmetric structure execution.[17]
Compute Overhead	High; requires GPU acceleration for graph convolutions and causal temporal convolutions.[9]	Low; efficient multi-core CPU parallelization.[3]
Key Parameters	Adjacency matrix distance σ [2], dilated causal kernel size k, dilation rate d.[33]	Learning rate η [40], colsample_bytree [36], random seed ϕ.[38]

Strict Time-Series Validation Protocol

Evaluating travel demand forecasting pipelines requires validation protocols that prevent temporal data leakage and replicate real-world deployment conditions.[3] Because traffic demand is highly autocorrelated, random K-fold cross-validation or out-of-fold strategies that ignore temporal ordering will leak future information into the training phase, leading to inflated validation metrics.[11]

To address this, the re-engineered pipeline implements a **Expanding Block Time-Series Validation Protocol**.[11]

The validation strategy splits the dataset into rolling temporal folds.[11] For each split, the model is trained on data up to day T_{train} and validated on the subsequent day $T_{\text{validate}} = T_{\text{train}} + 1$.[11] This rolling structure prevents future information leakage and simulates the daily retraining cycles used in production.[11]
The five validation splits are configured as follows:

Validation Split	Training Interval (Days)	Validation Window (Day)	Primary Target Objective
Split 1	Days 1 to 43	Day 44	Baseline performance evaluation on a standard weekday.
Split 2	Days 1 to 44	Day 45	Mid-week performance check and adaptation stability.
Split 3	Days 1 to 45	Day 46	Transition to weekend demand shifts (Day 46 represents Saturday).[18]
Split 4	Days 1 to 46	Day 47	Evaluation of weekend traffic behavior (Day 47 represents Sunday).[18]
Split 5	Days 1 to 47	Day 48	Final model adjustment before evaluating the test set on Day 49.[6]

To eliminate data leakage, any record from Day 49 is excluded from both the training and validation splits of this protocol.[6] When generating features for a target validation day T_{validate}, the pipeline restricts historical window lookbacks strictly to days $t < T_{\text{validate}}$.[11]

For example, when validating on Day 48, the spatial demand lag features ($Y_{\text{KNN\_Lag}}$) are computed using records from Day 47.[10] This temporal separation isolates the validation target, preventing leakage from future or contemporaneous records.[11]

Reproducibility, Verification, & Operationalization

To satisfy strict competition verification guidelines, the re-engineered pipeline is structured into a reproducible, modular execution script (predict.py).[6] This architecture ensures deterministic results by fixing random seeds and utilizing containerized dependencies, preventing leaderboard discrepancies.[6]

The prediction pipeline is structured as follows:

```
Python
import os
import sys
import argparse
import numpy as np
import pandas as pd
from sklearn.cluster import KMeans
from lightgbm import LGBMRegressor
from catboost import CatBoostRegressor

def set_reproducible_seeds(seed=42):
    """Fixes all environment and algorithmic seeds for exact reproducibility."""
    np.random.seed(seed)
    os.environ = str(seed)

def decode_geohash_to_coordinates(df, geohash_col='geohash'):
    """
    Decodes high-cardinality base-32 geohash strings
    into latitude/longitude centroids and 3D Cartesian coordinates.
    """
    # Alphabet map for Base32 decoding
    b32_map = {c: i for i, c in enumerate("0123456789bcdefghjkmnpqrstuvwxyz")}
```

```
    lats, lons = 
    for code in df[geohash_col]:
        # Perform de-interleaving and binary calculation
        # This implementation uses pygeohash-equivalent centroid calculation
        # for maximum numerical stability.
        lat_interval = [-90.0, 90.0]
        lon_interval = [-180.0, 180.0]
        is_even = True

        for char in code:
            val = b32_map[char]
            for i in range(4 - 1, -1, -1):
                bit = (val >> i) & 1
                if is_even:
                    mid = (lon_interval + lon_interval) / 2.0
                    if bit == 1:
                        lon_interval = mid
                    else:
                        lon_interval = mid
                else:
                    mid = (lat_interval + lat_interval) / 2.0
                    if bit == 1:
                        lat_interval = mid
                    else:
                        lat_interval = mid
                is_even = not is_even

        lats.append((lat_interval + lat_interval) / 2.0)
        lons.append((lon_interval + lon_interval) / 2.0)

    df['lat'] = lats
    df['lon'] = lons

    # Radians conversion and 3D Cartesian projection
    lat_rad = np.radians(df['lat'])
    lon_rad = np.radians(df['lon'])
    df['x'] = np.cos(lat_rad) * np.cos(lon_rad)
    df['y'] = np.cos(lat_rad) * np.sin(lon_rad)
    df['z'] = np.sin(lat_rad)
    return df

def generate_periodic_time_embeddings(df, timestamp_col='timestamp'):
    """
```

```
    Parses timestamp string to daily slots (0-95)
    and generates sine/cosine periodic embeddings.
    """
    # Parse timestamp "HH:MM" format
    time_split = df[timestamp_col].str.split(':', expand=True).astype(int)
    time_slots = (time_split[0] * 60 + time_split[1]) // 15

    df['time_slot'] = time_slots
    df['sin_time'] = np.sin(2.0 * np.pi * df['time_slot'] / 96.0)
    df['cos_time'] = np.cos(2.0 * np.pi * df['time_slot'] / 96.0)
    return df

def train_and_predict(train_path, test_path, out_path):
    """
    Executes advanced feature engineering, trains a multi-seed GBDT ensemble,
    and generates reproducible predictions.
    """
    set_reproducible_seeds(42)

    # Load dataset partitions
    train_df = pd.read_csv(train_path)
    test_df = pd.read_csv(test_path)

    # Temporal validation separation: Purge any Day 49 records from training
    # to eliminate chronological data leakage
    train_df = train_df[train_df['day'] != 49].reset_index(drop=True)

    # Process continuous spatial features
    train_df = decode_geohash_to_coordinates(train_df)
    test_df = decode_geohash_to_coordinates(test_df)

    # Process periodic temporal features
    train_df = generate_periodic_time_embeddings(train_df)
    test_df = generate_periodic_time_embeddings(test_df)

    # Apply spatial K-Means clustering
    kmeans = KMeans(n_clusters=15, random_state=42, n_init=10)
    train_df['region_id'] = kmeans.fit_predict(train_df[['x', 'y', 'z']])
    test_df['region_id'] = kmeans.predict(test_df[['x', 'y', 'z']])

    # Define modeling features
    features = ['x', 'y', 'z', 'sin_time', 'cos_time', 'region_id', 'NumberofLanes']
    target = 'demand'
```

```
    X_train = train_df[features]
    y_train = train_df[target]
    X_test = test_df[features]

    # Stochastic Retraining: Multi-seed average to mitigate first-mover bias
    seeds =
    test_preds = np.zeros(len(test_df))

    for seed in seeds:
        # LightGBM Regressor configuration
        lgb = LGBMRegressor(
            n_estimators=150
            learning_rate=0.05
            num_leaves=31
            colsample_bytree=0.3  # Low feature fraction to reduce collinearity bias
            random_state=seed
        )
        lgb.fit(X_train, y_train)
        test_preds += lgb.predict(X_test) / len(seeds)

    # Generate final output
    submission = pd.DataFrame({
        'Index': test_df['Index']
        'demand': np.clip(test_preds, 0.0, 1.0)  # Enforce physical range limits
    })
    submission.to_csv(out_path, index=False)
    print(f"Predictions successfully exported to {out_path}")

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Reproducible Prediction Pipeline")
    parser.add_argument("--train", required=True, help="Path to train.csv")
    parser.add_argument("--test", required=True, help="Path to test.csv")
    parser.add_argument("--out", required=True, help="Output path for submission.csv")
    args = parser.parse_args()

    train_and_predict(args.train, args.test, args.out)
```

Synthesis and Recommendations

The evaluation of the travel demand forecasting pipeline highlights the importance of matching spatial and temporal representation layers to the physical layout of the urban environment.[1]

- **Validation Security**: To ensure model stability, direct spatiotemporal lookup queries

must be replaced with generalizable feature engineering.[4] Enforcing a strict expanding block validation protocol ensures that validation metrics reflect actual out-of-sample forecasting performance.[11]

- **Feature Representation**: Converting raw, high-cardinality geohashes into 3D Cartesian coordinates and periodic sine/cosine temporal embeddings preserves continuous physical relationships.[11] This spatial-temporal consistency helps both neural networks and gradient boosting ensembles capture recurring patterns.[9]
- **Modeling Strategy**: For operational environments with strict latency limits, a Heterogeneous GBDT Ensemble using **Multi-Seed Averaging** is recommended.[3] This approach offers competitive accuracy with low compute overhead, and stochastic retraining helps mitigate first-mover bias under high feature collinearity.[3]

By implementing these structural updates, the travel demand pipeline is secured against spatiotemporal leakage and optimized for robust, real-time prediction.[3]

Works cited

1. Spatio-temporal multi-graph networks for demand forecasting in online marketplaces - Amazon Science, accessed June 5, 2026, https://www.amazon.science/publications/spatio-temporal-multi-graph-networks-for-demand-forecasting-in-online-marketplaces
2. An Efficient Short-Term Traffic Speed Prediction Model Based on Improved TCN and GCN, accessed June 5, 2026, https://www.mdpi.com/1424-8220/21/20/6735
3. Enhancing network resource management through machine learning for spatio-temporal beam-level traffic forecasting - ITU, accessed June 5, 2026, https://www.itu.int/dms_pub/itu-s/opb/jnl/S-JNL-VOL6.ISSUE4-2025-A27-PDF-E.pdf
4. DATA LEAK IN Gridlock Hackathon 2.0 : r/Btechtards - Reddit, accessed June 5, 2026, https://www.reddit.com/r/Btechtards/comments/1ts5azu/data_leak_in_gridlock_hackathon_20/
5. DATA LEAK IN Gridlock Hackathon 2.0 : r/NSUT_Delhi - Reddit, accessed June 5, 2026, https://www.reddit.com/r/NSUT_Delhi/comments/1ts5by1/data_leak_in_gridlock_hackathon_20/
6. Bannysukumar/Gridlock-Hackathon-2.0 - GitHub, accessed June 5, 2026, https://github.com/Bannysukumar/Gridlock-Hackathon-2.0
7. FlipkartGridlockML.ipynb
8. Spatiotemporal Attention Networks for Traffic Demand Prediction - CEUR-WS.org, accessed June 5, 2026, https://ceur-ws.org/Vol-3304/paper16.pdf
9. ADSTGCN: A Dynamic Adaptive Deeper Spatio-Temporal Graph Convolutional Network for Multi-Step Traffic Forecasting - PMC, accessed June 5, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC10422259/
10. Understanding Supply & Demand in Ride-hailing Through the Lens of Grab Data - Medium, accessed June 5, 2026,

https://medium.com/data-science/understanding-supply-demand-in-ride-hailing-through-the-lens-of-grab-data-23e24224547f
11. kornesh/grab-challenge: Grab AI Challenge: Traffic Management - GitHub, accessed June 5, 2026, https://github.com/kornesh/grab-challenge
12. aiforsea computer | Grab SG, accessed June 5, 2026, https://www.grab.com/sg/aiforsea/aiforsea-computer/
13. Traffic management - travel demand forecast - Kaggle, accessed June 5, 2026, https://www.kaggle.com/code/winnie19900705/traffic-management-travel-demand-forecast
14. Geohash - Wikipedia, accessed June 5, 2026, https://en.wikipedia.org/wiki/Geohash
15. Geohashing for Developers: Efficiently Encoding and Querying Locations | by Frank Edomaruse | Medium, accessed June 5, 2026, https://medium.com/@franksagie1/optimizing-location-based-apps-with-geohash-369738597b8c
16. This is how the Grab app knows exactly where users are | Inside Grab, accessed June 5, 2026, https://www.grab.com/sg/inside-grab/stories/grab-geohashing-location-grid/
17. Ensemble Learning Methods: A Comprehensive Guide to Combining Models for Superior Performance | by HairuFan | Medium, accessed June 5, 2026, https://medium.com/@hairufan/ensemble-learning-methods-a-comprehensive-guide-to-combining-models-for-superior-performance-9d0ac06e0b65
18. Grab Traffic Mgmt - Kaggle, accessed June 5, 2026, https://www.kaggle.com/code/samsonleegh/grab-traffic-mgmt
19. python-geohash - GeohashReference.wiki - Google Code, accessed June 5, 2026, https://code.google.com/archive/p/python-geohash/wikis/GeohashReference.wiki
20. Grab Traffic Demand Forecasting - Kaggle, accessed June 5, 2026, https://www.kaggle.com/code/mahadir/grab-traffic-demand-forecasting
21. geohash/README.md at main - GitHub, accessed June 5, 2026, https://github.com/nesterenko-kv/geohash/blob/main/README.md
22. GitHub - vinsci/geohash: Python module to decode/encode Geohashes to/from latitude and longitude. See http://en.wikipedia.org/wiki/Geohash · GitHub, accessed June 5, 2026, https://github.com/vinsci/geohash/
23. Spatio-Temporal Graph Convolutional Networks for EV Charging Demand Forecasting Using Real-World Multi-Modal Data Integration - arXiv, accessed June 5, 2026, https://arxiv.org/html/2510.09048v3
24. What is the k-nearest neighbors algorithm? - IBM, accessed June 5, 2026, https://www.ibm.com/think/topics/knn
25. Team Affine Anonymous wins Flipkart's Gridlock Hackathon on solutions for Bengaluru's traffic menace, accessed June 5, 2026, https://affine.ai/newsroom-post/team-affine-anonymous-wins-flipkarts-gridlock-hackathon-on-solutions-for-bengalurus-traffic-menace/
26. What is k-Nearest Neighbor (kNN)? - Elastic, accessed June 5, 2026, https://www.elastic.co/what-is/knn

27. How KNN Solves the Spatial Density Problem for Large-Scale Proximity Analysis, accessed June 5, 2026, https://wherobots.com/blog/scaling-spatial-analysis-how-knn-solves-the-spatial-density-problem-for-large-scale-proximity-analysis/
28. Travel Time Prediction Using Machine Learning Algorithms: Focusing on k-NN, LSTM, and Transformer - The Open Transportation Journal, accessed June 5, 2026, https://opentransportationjournal.com/VOLUME/18/ELOCATOR/e26671212356139/FULLTEXT/
29. Seeing Patterns in the Chaos: How Spatio-Temporal Kriging is Revolutionizing Prediction | by Everton Gomede, PhD | The Modern Scientist | Medium, accessed June 5, 2026, https://medium.com/the-modern-scientist/seeing-patterns-in-the-chaos-how-spatio-temporal-kriging-is-revolutionizing-prediction-188ecf646d9f
30. Kriging Interpolation Explanation - Columbia University Mailman School of Public Health, accessed June 5, 2026, https://www.publichealth.columbia.edu/research/population-health-methods/kriging-interpolation
31. Pratik Nag - Spatio-Temporal DeepKriging for Probabilistic Interpolation and Forecasting, accessed June 5, 2026, https://pratiknag.com/files/Presentation_spatio_temporal_deepkriging_interview.pdf
32. Using Kriging Technique to Interpolate and Forecasting Temperatures Spatio-Temporal Data - European Journal of Pure and Applied Mathematics, accessed June 5, 2026, https://www.ejpam.com/ejpam/article/view/4613/1364
33. Distributed Short-Term Traffic Flow Prediction Based on Integrating Federated Learning and TCN - IEEE Xplore, accessed June 5, 2026, https://ieeexplore.ieee.org/iel8/6287639/10380310/10705160.pdf
34. Spatio-Temporal Joint Graph Convolutional Networks for Traffic Forecasting, accessed June 5, 2026, https://asc.xmu.edu.cn/storage/file/202406/caf74d1a08153b76e17c67d4454d03d3.pdf
35. CEEMDAN-CatBoost-SATCN-based short-term load forecasting model considering time series decomposition and feature selection - Frontiers, accessed June 5, 2026, https://www.frontiersin.org/journals/energy-research/articles/10.3389/fenrg.2022.1097048/full
36. Multi-Seed Ensemble of Graph Attention Networks and Gradient Boosting for pIC50 Prediction: An Honest Out-of-Fold Benchmark on C - ChemRxiv, accessed June 5, 2026, https://chemrxiv.org/doi/pdf/10.26434/chemrxiv.15001489
37. Ensemble Methods in Machine Learning | by Shashank Bhatnagar - Medium, accessed June 5, 2026, https://medium.com/@shashank25.it/ensemble-methods-in-machine-learning-2d4cc7513c77
38. First-Mover Bias in Gradient Boosting Explanations: Mechanism, Detection, and

Resolution - arXiv, accessed June 5, 2026, https://arxiv.org/pdf/2603.22346
39. First-Mover Bias in Gradient Boosting Explanations: Mechanism, Detection, and Resolution, accessed June 5, 2026, https://arxiv.org/html/2603.22346v2
40. Strength in Numbers: Ensembling Models with Bagging and Boosting, accessed June 5, 2026, https://towardsdatascience.com/strength-in-numbers-ensembling-models-with-bagging-and-boosting/
41. Spatial–Temporal-Correlation-Constrained Dynamic Graph Convolutional Network for Traffic Flow Forecasting - MDPI, accessed June 5, 2026, https://www.mdpi.com/2227-7390/12/19/3159
42. Gridlock Hackathon 2.0 | Flipkart × Bengaluru Traffic Police - HackerEarth, accessed June 5, 2026, https://gridlock2point0.hackerearth.com/